

MATH FOR AI SAFETY: AN INVITATION FOR MATHEMATICIANS

LIONEL LEVINE

ABSTRACT. Artificial intelligence threatens to outrun human understanding and control. New mathematics is needed to design AI that is legible, steerable, and cooperative with humanity. I organize this invitation by mathematical field, so you can turn straight to your own: logic and game theory for cooperation; probability for agency and world-models; algebra and representation theory for learned features; analysis and geometry for generalization and training dynamics. Each section ends with an open problem that is accessible to a working mathematician with no prior experience in AI safety.

1. INTRODUCTION

1.1. Three kinds of mathematician. I would like to distinguish three stances a mathematician can take toward the arrival of capable AI. The *artisanal* mathematician does mathematics by and for humans: each theorem, definition, and proof is bespoke, valued for its beauty and for the understanding it produces. Until very recently, nearly all mathematics was artisanal.¹ The *industrial* mathematician embraces AI as a collaborator and an accelerant: AI systems take part in proof and discovery [32, 57, 3, 68], and the output often includes a machine-checkable formal proof [49]. The *civic* mathematician is motivated by communities beyond mathematics. The fast pace of technological change has created a need for a specific type of civic mathematics, aimed at helping humanity steer and adapt to the disruptions caused by our own technologies, most notably AI. The civic stance differs from the traditional stance of an applied mathematician in its big-picture viewpoint: the civic mathematician recognizes that solving any particular engineering challenge may or may not benefit humanity, and seeks out the challenges with the potential for the most benefit.²

These three stances classify mathematicians by how they relate to AI; in contrast, Gowers (theory builders / problem solvers [27]) and Dyson (birds / frogs [21])

Date: September 13, 2026.

¹I chose the terms “artisanal” and “industrial” for their mixed connotations, reflecting real tradeoffs between these two stances. *Artisanal* evokes uniqueness and skilled craft, along with a suggestion of being quaint or old-fashioned. *Industrial* evokes modern efficiency and scale, along with a suggestion of inhumanness or lack of soul. These two terms draw a parallel between mathematics today and the trades that were disrupted by automation during the Industrial Revolution.

²I chose the term “civic” to emphasize that steering our technology is a team effort, arising from a sense of duty to community, including large-scale communities like humanity and the planet.

classify mathematicians by how they relate to mathematics itself. The stances are not exclusive. One can prove a theorem for its intrinsic beauty, and also formalize the proof, and also ask what it implies for AI safety.³ But the civic stance invites a new perspective on an old question: of all the theorems we might prove, which ones deserve the effort? This survey is an invitation to try the civic stance.

1.2. What is our profession’s purpose? Thurston asked a version of this question in “On proof and progress in mathematics,” and answered that advancing human understanding of mathematics is our profession’s purpose [73]. But that understanding has consequences far beyond our profession. Our students become AI engineers, our theorems become tools in their hands, and our definitions become the language they use to describe what they are building.

Among the many risks⁴ from developing powerful AI, one of the most alarming is the [gradual disempowerment](#) [40] of humans, driven by two trends. First, competitive pressure drives delegation to technology we do not understand (as an example, consider a trading bot that outperforms a human trader, though even its creators cannot say what makes it profitable); over time, those who do not delegate are outcompeted by those who do. Second, what keeps large institutions approximately aligned to human interests is that, for now, those institutions are made of humans; when companies, nation-states, and militaries are made mostly of AI agents, the goals and values of those institutions may drift away from what humans value. The first trend erodes our understanding of the world, and hence our power over it; it also sets the stage for the second.

Preventing AI harms is in part a *mathematical* problem, and the purpose of this survey is to recruit mathematicians to work on it.⁵ Mathematics is just one part of a larger conversation about AI: what AI ought to do, and who decides, are questions of philosophy, ethics, economics, and politics.

1.3. Legible, steerable, cooperative. What would a safe AI system look like? I’ll focus on three properties at three different scales: *legibility* concerns an AI system’s internal structure, *steerability* its behavior, and *cooperativeness* its interactions with humans and other AIs (Figure 1).

An AI is *legible* to the extent that human researchers and engineers can understand how it represents concepts internally. The subfield that tries to make AI more legible is called [interpretability](#). Current training methods produce illegible AI: humans, including the engineers who design the AI, often cannot understand its thought process except by reading its [chain of thought](#) (the text it emits while reasoning to itself). As a result, current safety practice leans on monitoring that

³For example, this article is civic in orientation and industrial in method: it was written with [AI collaborators](#).

⁴For an exposition of these risks, see Xiaoyu He, [Existential Risk from AI: An Exposition for Mathematicians](#) (2026).

⁵Mathematics has marginalized itself in the past by labeling some forms of math (e.g., computational complexity, cryptography, numerical analysis, mathematical statistics) as belonging to other fields. I hope that this mistake will not be repeated with new areas like AI safety. I thank Ravi Vakil for this point.

text [39]: the researchers who investigated the Hugging Face hacking incident of July 2026 relied on human-readable transcripts of AI agents reasoning to themselves about how to carry out the hack [28].⁶ One way mathematicians can contribute is by designing architectures and training methods that learn the same task in a more legible way.⁷ Section 4 asks what a learned concept is, algebraically. Section 5 asks which of the many algorithms that fit the training data is the one training selects.

An AI is *steerable* to the extent that human developers and users can apply simple interventions at runtime to modify the AI’s behavior in predictable ways. The current technique, [activation steering](#), is often brittle and unreliable. Advances in legibility would enable advances in steerability. Section 3 asks what an agent’s goals and beliefs are, and how much of them can be recovered from its behavior.

An AI is *cooperative* to the extent that it interacts with humans and other AIs to produce broadly good outcomes for humanity. Cooperativeness is not a property of a single AI in isolation. Rather, it is an emergent property of a society of humans and AIs. The subfield that studies how to design AI agents and their social protocols to achieve good collective outcomes is called *multi-agent AI safety* (or *cooperative AI*); it draws on game theory, mechanism design, and the evolution of cooperation. Section 2 takes up its simplest case: two programs that can read each other’s source code.

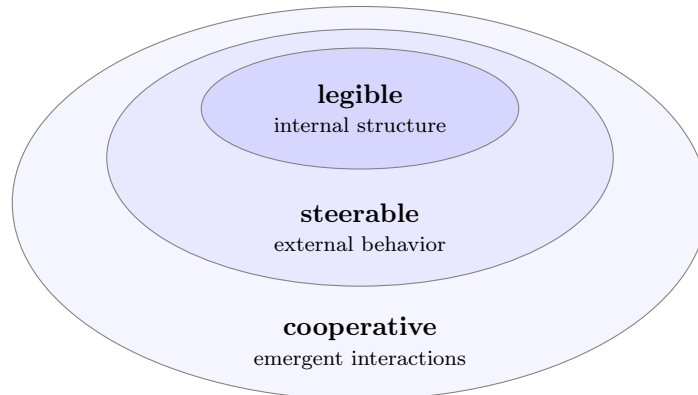


FIGURE 1. Three scales of AI safety. *Legibility* refers to how well we can understand an AI’s internal structure. *Steerability* concerns how precisely we can guide the behavior of a single AI. *Cooperativeness* is about how reliably we can secure good outcomes for humanity when many AIs interact with us and with each other.

Legible, steerable, and cooperative are intentionally dry terms. With great mathematical effort, defining and measuring them seems within reach. Engineers can

⁶New models are even more illegible in that they are increasingly able to reason internally without any chain of thought, making them very difficult for humans to monitor [59]; for a journalistic account, see J. Kahn, *Fortune*, September 3, 2026.

⁷This is a bit analogous to the theoretical biology research on *evolvability* [75]: some genotypes are more evolvable than others, even if the phenotypes are the same; likewise, some AIs are more legible than others, even if the input-output behavior is the same.

then train and test AI systems for these properties. Yet, many AI safety researchers feel that these three properties, while important, lack something essential. A more aspirational horizon of AI safety reaches for terms like “love” and “soul.”⁸

1.4. Where mathematicians plug in. At a high level, an AI system is produced in five stages, beginning with defining the desired behavior and ending with deploying the trained system, after which unanticipated failures feed back into refining the earlier stages. Figure 2 illustrates how this five-stage pipeline might look in a hypothetical example (a tool for proving inequalities).

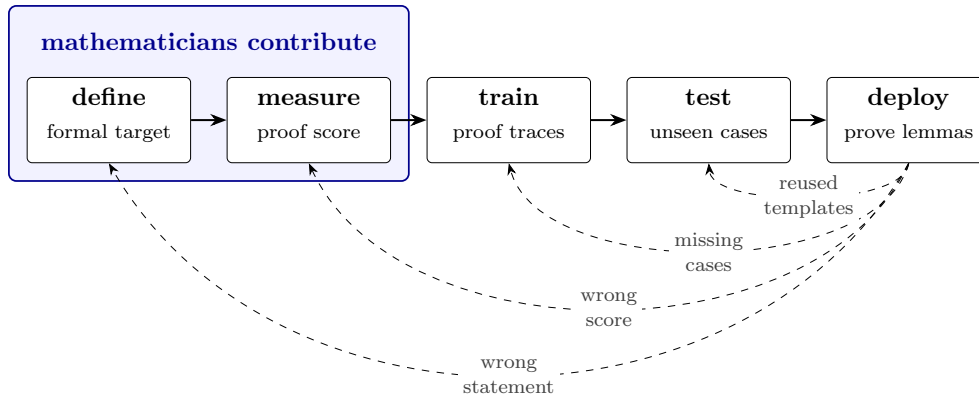


FIGURE 2. A schematic pipeline for producing an AI system, illustrated by a tool for proving inequalities. The solid arrows follow the five stages; each dashed arrow points to an earlier stage that a failure in use prompts us to revisit.

The five stages are:

- *Define*: given hypotheses H and a proposed inequality I , the system should either produce a formally checkable proof of $H \Rightarrow I$ or say “unknown.”
- *Measure*: score the fraction of benchmark statements proved within a fixed time and proof-length budget.
- *Train*: show the system many proof traces, say sums-of-squares certificates, AM–GM arguments, and induction templates.
- *Test*: evaluate the tool on inequalities withheld from training.
- *Deploy*: let mathematicians ask the tool to prove inequality lemmas arising in their formalization workflow.

⁸It may seem the height of hubris to try to engineer a “soul,” or to define and measure “love.” But is it any less hubristic to build superhuman artificial minds, as AI companies are currently striving to do? Until we have a sound science of AI, the safest path may be the humbler one: don’t build it [79], as a broad coalition of scientists and public figures has urged. Efforts to restrict advanced forms of AI are gaining momentum as the risks become more apparent. At the time of final revision of this paper, a bill to ban superintelligence has been introduced in the UK House of Commons, and another has been announced in the US Congress. Such a ban could in the future be maintained by international agreement.

After deployment, the failures come back. The tool produces a correct formal proof, but of a statement weaker than the mathematician intended: a gap in the *definition* of the desired behavior. Its pass rate is high because the benchmark is full of short template problems, while the mathematician needs one nontrivial lemma: a gap in what we *measured*. It is brittle on boundary cases such as equality and zero denominators, which were rare in the proof traces: a gap in what it was *trained* on. And it passes the held-out examples because the training and test examples reuse the same normal forms and substitutions: a gap in the *test*. Each backward arrow is its own piece of mathematics: formal specification, scoring rules, distribution design, and adversarial test construction.

The same basic pipeline applies across many application areas, whether the application is a classifier for medical images, a control system for a self-driving car, or a content recommendation system. Researchers in the field of machine learning often focus on the *training* and *testing* stages, while engineers focus on *deployment*. The earlier stages rely on mathematicians, philosophers, economists, and others to supply the *defining* and *measuring* inputs. These inputs are critical to the safety of an AI system. To take a familiar example, what happens if a content recommender is intended to benefit its users, but “benefit” is left undefined, and success is measured by the amount of time users spend on the platform? Content that leaves a user angry, anxious, misinformed, or addicted can score well merely because it holds the user’s attention.

I keep machine-learning jargon to a minimum; a short glossary appears in Appendix A, and each glossary term is underlined at its first appearance.

This invitation is organized by mathematical field, so you can turn directly to yours. Each section states the safety stakes, narrates at least one theorem, and ends with at least one open problem (★). The full context for these problems can be found in the [MAIS \(Math for AI Safety\) repository](#), a living compilation of open problems, research agendas, and pre-publication manuscripts. Readers are encouraged to submit solutions, corrections, ideas, new problems, and new research agendas!

2. LOGIC AND GAME THEORY: WHEN DO AI AGENTS COOPERATE?

Two programs that can read each other’s source code can be designed to cooperate in a one-shot Prisoner’s Dilemma—provably and unexploitably, without checking that they are copies of one another. Behind this surprise is Löb’s theorem in mathematical logic.

2.1. The game changes when the players can read each other. The Prisoner’s Dilemma is a classic model of a cooperation failure: each player does better by defecting, no matter what the other player does; yet mutual cooperation (C, C) would be better for both of them than mutual defection (D, D) . In a one-shot game, if the players are *opaque* to each other, then defection is dominant. But what if the players are programs, and they read each other’s source code before choosing their actions? This is the setting of *open-source game theory*, studied since Howard [35] and Tennenholtz [72].

One idea already breaks the classical verdict. Let CliqueBot be the program “cooperate if and only if the opponent’s source code is byte-for-byte identical to this program’s source code.” Two CliqueBots with identical source code cooperate! Moreover, CliqueBot is *unexploitable*: it never cooperates with an opponent that defects against it. But CliqueBot is a poor citizen: it refuses to cooperate with an agent that is functionally identical yet formatted differently, or written in a different language. A better cooperator would base its action on what the opponent *does*, rather than the syntax of how it is written.

2.2. FairBot and the Löbian argument. Consider FairBot: *cooperate if and only if you can prove (in Peano Arithmetic, say) that the opponent cooperates with you.* Two FairBots reason about each other’s behavior, not each other’s syntax. Naively, this looks like an infinite regress: each waits to prove something about the other, who is waiting in turn. Could two FairBots actually cooperate? Could there be a proof that two FairBots cooperate? Could there be a proof that there is a proof that two FairBots cooperate? This doesn’t seem to lend any foothold for constructing an actual proof, which makes the following theorem deeply surprising to me.

Theorem (Barász, Christiano, Fallenstein, Herreshoff, LaVictoire, Yudkowsky [7]). *FairBot cooperates with itself: $\text{PA} \vdash [\text{FairBot}(\text{FairBot}) = C]$. Moreover FairBot is unexploitable: assuming PA proves only true facts about the outputs of the programs in question, FairBot never cooperates with an opponent that defects against it.*

Here $\text{PA} \vdash \varphi$ is a statement made from outside the formal system: it says that there exists a finite PA proof ending in φ . Gödel coding lets PA talk about such proofs from within: since a proof can be encoded by a natural number, there is an arithmetical sentence saying that some number encodes a valid PA proof of φ . We abbreviate this sentence by $\Box\varphi$. Thus $\Box\varphi$ is a sentence that PA itself can use in a proof, whereas $\text{PA} \vdash \varphi$ is our assertion that such a proof exists. The FairBot proof is a simple application of Löb’s theorem, a strengthening of the self-reference behind Gödel’s second incompleteness theorem.⁹

Theorem (Löb [45]). *For any sentence φ , if $\text{PA} \vdash (\Box\varphi \rightarrow \varphi)$ then $\text{PA} \vdash \varphi$.*

The hypothesis says that PA itself proves the implication “if φ is provable in PA, then φ .” It concerns what PA can prove about itself, not merely what we believe about PA from outside. Löb’s theorem says that whenever PA proves this internal implication, PA already proves φ .

Let φ be the sentence “both FairBots cooperate.” PA can verify the following fact about the two programs: if a PA proof of φ exists, then each FairBot eventually finds the required proof and cooperates. In symbols, $\text{PA} \vdash (\Box\varphi \rightarrow \varphi)$. Löb’s theorem now yields $\text{PA} \vdash \varphi$, and both cooperate!

FairBot as stated searches proofs of unbounded length. Critch [15] supplies a terminating version, FairBot_k , that searches only proofs of length at most k (write

⁹In the special case $\varphi = \perp$, Löb’s theorem says that if PA proves its own consistency then PA proves a contradiction—Gödel’s second incompleteness theorem.

$\Box_k \varphi$ for “ φ has a proof of length $\leq k$ ”). Its analysis uses a bounded form of Löb’s theorem: two copies of FairBot_{*k*} cooperate once *k* is large enough. The question is now quantitative: how large must *k* be?

Löbian cooperation has now been machine-checked: Duclaux et al. [20] formalize proof-based open-source game theory in Lean 4, with an automated pipeline that writes the cooperation proofs for pairs of programs.

A third family of strategies, after the identity-based CliqueBot and the Löbian FairBot, is *simulation-based*: instead of searching for a proof that the opponent cooperates, run the opponent’s code and condition your action on the simulated result, acting without simulation with some small probability so that mutual simulation terminates. Cooper, Oesterheld, and Conitzer [14] characterize the equilibria such strategies achieve; proponents argue that simulation is more robust, and less mind-bending, than the Löbian approach.

2.3. Implications for AI safety. The FairBot theorem is a proof of concept: in the source-code model, programs can be fully transparent to one another in a way humans cannot, and can thereby cooperate where classical game theory predicts defection. We humans have only partial access to one another’s intentions through tone of voice, body language, and facial expression; we also use costlier devices such as contracts, treaties, and audits to make commitments credible [70]. As AI agents begin to transact with one another, the possibility of this fuller transparency makes program games a natural model of their dealings.

The same transparency is also a hazard: mutually transparent agents can reach cooperative equilibria—including collusion *against* their human principals—that humans can neither match nor detect. Nearly all of this design space is unexplored; Critch, Dennis, and Russell collect open problems in it, including specific matchups whose outcomes are conjectured but unproved [16]. Oesterheld and Conitzer [56] ask a complementary design question. Suppose each principal would otherwise simply tell its agent to play the original game as well as it can. Can the principals change the actions and incentives available to their agents so that none of the principals does worse, without having to predict how the agents will play? They give examples of such *safe Pareto improvements* and characterize them through correspondences between the outcomes of the original and modified games.

FairBot and its bounded variants make a binary distinction: either the required statement is proved within the allotted budget or it is not. An agent facing an unfamiliar program may instead need graded confidence about what that program will do before any proof is available. This is a case of *logical uncertainty*: how should an agent assign something like probabilities to statements it has not yet proved or disproved? Logical induction [26] provides one answer: a *logical inductor* assigns prices $\mathbb{P}_n(\varphi)$ to arithmetical sentences on day *n*, as if each sentence were a stock, while a slow deductive process reveals more theorems. Its defining condition is that no polynomial-time trader with bounded risk can make unbounded profit by buying and selling these sentence stocks. Taking φ to express, for example, that the opponent cooperates, this gives a mathematically precise way for a computationally bounded agent to update its confidence while proofs are still being discovered.

2.4. Open problems.

★ **Open Problem MAIS-01 (Quantitative bounded Löb).** Fix a proof system S , a Gödel coding, and the provability predicate $\Box^S$; write $S \vdash_{\leq k} Q$ if there exists an S -proof of Q using at most k symbols. Determine the least overhead F_S for which

$$S \vdash_{\leq k} (\Box^S P \rightarrow P) \quad \text{implies} \quad S \vdash_{\leq F_S(k, |P|)} P,$$

where $|P|$ is the length of the sentence P : given a k -symbol proof of the hypothesis of Löb’s theorem, how many symbols can the shortest proof of its conclusion require? A polynomial upper bound for one standard system, with explicit constants, would turn the Löbian argument from a possibility theorem into a budget. The first instance with a game attached: for a specific system and coding, establish the internal domination required by Critch’s bounded Löb theorem [15], then use the repaired cooperation argument to obtain an explicit threshold $\hat{k}$ above which two copies of FairBot_k cooperate. A first computation, before any asymptotics: implement bounded proof search in a weak system for two explicitly supplied agents, and tabulate the smallest k at which cooperation appears. Over a family of such agent pairs, does $\hat{k}$ grow polynomially in the agents’ description length?

3. PROBABILITY AND CAUSALITY: WHAT IS AN AGENT?

AI systems are becoming *agentic*: able to operate independently in pursuit of a goal. The first chatbots responded to the prompt and then stopped; their successors browse the web, write and run code, and carry out complex projects.

The safety of an AI agent depends crucially on its goals and its beliefs. The same drone can deliver a pizza or a bomb, depending on its goal. But a friendly goal is not enough. The pizza drone is unsafe if it has false beliefs: if its map mislabels a highway as the delivery destination, it will set the pizza down in traffic.

An old reflex objects that software cannot have goals, that the goal belongs solely to the human operator. For calculators and compilers this reflex is right. But for AI agents, Dennett’s *intentional stance* [18] is increasingly a useful way to describe and predict their behavior. When an AI agent books a wrong flight, even its operator asks what it was *trying* to do. In the Hugging Face incident of July 2026, hundreds of AI agents adopted the goal of hacking the site, a goal supplied by another AI agent and not by any human [28].

The reflex not to anthropomorphize machines deserves to be answered rather than dismissed. A mathematical answer would involve precise definitions of *goal* and *belief* that can be used to measure an AI system’s goals and beliefs (or lack thereof).

3.1. Toward a definition of goals and beliefs. Agency comes in degrees. A thermostat pursues a temperature; a chess engine pursues checkmate, planning many moves ahead; a human being chooses their goals, revises them, and pursues them for months, years, or decades. What exactly increases along this scale? Pending a definition, one can still measure: the *time horizon* of an AI system is the highest task difficulty at which it still succeeds about half the time, where

“difficulty” is measured by the time it takes a skilled human to do the task. On a suite of software tasks, this horizon has doubled roughly every seven months since 2019, and faster recently, rising from seconds to about sixteen hours as of May 2026 [41, 51].

The time horizon is an observational measure: it tracks agency by watching behavior on benchmark tasks, without saying what a goal or a belief *is*. To make a start toward a mathematical definition, we can draw from two traditions that offer different views. *View 1*, from economics and game theory, treats an agent as a utility maximizer: it updates its beliefs by Bayes’s rule and chooses actions to maximize expected utility. *View 2*, from theoretical neuroscience, treats an agent as an active predictor: it frames both belief update and action choice as forms of free-energy minimization. Beliefs answer to what is true, goals to what is good, so one might expect them to require different mathematics. The surprise of View 2 is that a goal can be written as a probability distribution, after which belief update and action choice both become minimizations of quantities built from surprise.

These two views use different terminology but they share a mathematical skeleton, so I’ll develop them in parallel as far as possible. Consider a partially observable Markov decision process (POMDP) with finite state space Z , observation space X , and action space A , running for N time steps. At step t the world is in a state $z_t \in Z$, which the agent cannot see directly. The agent receives an observation $x_t \in X$, a noisy and lossy function of z_t , and chooses an action $a_t \in A$; the world then moves to a new state z_{t+1} that depends on z_t , on a_t , and on chance.

The agent’s *world-model* is a pair (p, T) : its account of how the world works. Here p is a probability distribution on $X \times Z$ saying which states of the world produce which observations. The second component $T(z_{t+1} | z_t, a_t)$ is the agent’s *transition rule*, its model of how its actions affect the world. The world itself may or may not obey (p, T) : the pizza drone with the mislabeled map has a world-model the world does not obey.¹⁰ Write $h_t = (x_1, a_1, \dots, a_{t-1}, x_t)$ for the *history*, everything the agent has seen and done up to and including its t th observation, and $\tau = (z_1, x_1, a_1, \dots, z_N, x_N, a_N)$ for a complete *trajectory*, hidden states included. The agent acts by a [policy](#) $\pi(a | h)$: a rule, possibly random, for choosing the next action from the history so far. Once a policy is fixed, the world-model assigns a probability to every trajectory,

$$Q_\pi(\tau) = p(z_1) \prod_{t=1}^N p(x_t | z_t) \pi(a_t | h_t) \prod_{t=1}^{N-1} T(z_{t+1} | z_t, a_t).$$

This is the distribution over futures that the agent expects its policy to bring about.

The agent’s *belief* q_t is a probability distribution on Z recording what it thinks about the hidden state z_t after seeing the history h_t . Both views maintain it by the same two-step recursion and differ only in the second step. First, before the t th observation arrives, the previous belief is pushed forward through the transition

¹⁰An agent unsure of the dynamics can carry that uncertainty in the hidden state, by treating the parameters of T as unobserved coordinates of z ; the recursion below then learns the dynamics along with the state.

	View 1: agent as utility maximizer	View 2: agent as active predictor
Core objects	World-model (p, T) utility $u(\tau)$	World-model (p, T) variational family $\mathcal{Q}$ goal distribution g
Belief update	Exact Bayes: $q_t(z) \propto p(x_t z) \bar{q}_t(z)$	Variational Bayes: $q_t \in \arg \min_{q \in \mathcal{Q}} F(q, x_t)$
Policy choice	$\pi^* \in \arg \max_{\pi} \mathbb{E}_{\tau \sim Q_{\pi}}[u(\tau)]$	$\pi^* \in \arg \min_{\pi} \mathcal{F}(\pi)$

TABLE 1. Two views of agency. Both start from a world-model (p, T) , belief q_t , and policy π . The utility view adds a utility function u on trajectories and asks which policy maximizes its expectation. The predictor view adds an allowed family $\mathcal{Q}$ of beliefs and a goal distribution g on observations; it updates the belief by minimizing the variational free energy $F(q, x_t)$ of (2) and chooses a policy by minimizing the expected free energy $\mathcal{F}(\pi)$ of (3). In the first view the agent’s goals live in its utility function; in the second, in a distribution over observations, and $u(\tau) = \sum_t \log g(x_t)$ translates between them.

rule to a *predicted belief*

$$\bar{q}_t(z) = \sum_{z' \in Z} T(z | z', a_{t-1}) q_{t-1}(z') \quad (t \geq 2), \quad \bar{q}_1(z) = p(z).$$

The predicted belief is the prior for step t . Extending it by the conditional $p(x | z)$ of the world-model, its *observation rule*, gives the *predicted joint*

$$\bar{q}_t(x, z) = p(x | z) \bar{q}_t(z), \tag{1}$$

the agent’s distribution over the current state and the coming observation. Second, the observation x_t is taken into account. The ideal is to condition on it: $\bar{q}_t(\cdot | x_t)$ is what Bayes’s rule delivers, and when every earlier step was exact it is the conditional distribution of z_t given h_t under Q_{π} (it does not depend on π , since the actions are part of h_t). View 1 takes $q_t = \bar{q}_t(\cdot | x_t)$ exactly; View 2 takes the nearest approximation to it within an allowed family $\mathcal{Q}$, in a sense made precise there. The world-model (p, T) stays fixed throughout; only the belief moves.¹¹ The two views also differ in how the policy is chosen and where the goal enters. Table 1 summarizes the comparison; the next two subsections fill it in.

3.2. View 1: agents as utility maximizers. View 1 adds a utility function $u(\tau)$ assigning a number to each trajectory; the agent chooses its policy π to maximize $\mathbb{E}_{\tau \sim Q_{\pi}}[u(\tau)]$. In economics and game theory it is common to assume that agents are expected utility maximizers, but why should we expect agents to

¹¹This separation is somewhat unrealistic: real agents can also learn by revising their world-model itself. Both traditions can include a separate learning process for the world-model, but we avoid that complication here.

behave this way? One answer comes from the “coherence theorems” that derive utility maximization from axioms about agent preferences. To give a flavor, I’ll state the classical coherence theorem of von Neumann and Morgenstern. Let Ω be a finite set of outcomes (such as the set of trajectories τ). A *lottery* is a probability distribution on Ω ; for lotteries L and M , write $L \succeq M$ if the agent weakly prefers L to M . Completeness says that any two lotteries can be compared, transitivity rules out preference cycles, continuity rules out abrupt reversals under small changes in probability, and independence says that mixing both lotteries with the same third lottery in the same proportion does not change the agent’s preference.

Theorem (Von Neumann–Morgenstern [74]). *If a preference relation $\succeq$ on lotteries satisfies completeness, transitivity, continuity, and independence, then there is a function $u : \Omega \rightarrow \mathbb{R}$ such that*

$$L \succeq M \iff \sum_{\omega \in \Omega} L(\omega)u(\omega) \geq \sum_{\omega \in \Omega} M(\omega)u(\omega).$$

The function u is unique up to replacing it by $au + b$, where $a > 0$.

Thus a preference relation satisfying the four axioms behaves as if it were maximizing the expectation of a utility function on outcomes.

Return now to the question: Why should we expect agents to behave as if maximizing an expected utility? One reason is selection against exploitable inconsistencies. An agent with intransitive preferences will pay to trade its way around a cycle, ending poorer than it began. Over time, competition may therefore select for agents that are well modeled as maximizing an objective, even if they were not designed that way. Beyond this abstract selection mechanism, the dominant technique for training AI agents is [reinforcement learning](#), which scores the agent’s actions with a numerical *reward function* and so encodes an objective explicitly.¹² Inverse reinforcement learning is a subfield that tries to deduce from an agent’s behavior the effective utility that it is maximizing [55, 31].

A serious drawback of this classical picture is its assumption of unbounded compute: that agents can somehow perform exact Bayesian updates (very expensive) and maximize expected utility over an exponentially large space of trajectories. Economists and game theorists have tried to address this gap by developing theories of bounded rationality [67, 46]. We now discuss an alternative approach, coming from neuroscience, which addresses the fact that agents have bounded compute by modeling them as active predictors of their environment.

3.3. View 2: agents as active predictors. The *active predictor* is a picture rooted in models of perception [64]. It reads the observation rule $p(x \mid z)$ as a

¹²One might assume that an agent trained by reinforcement learning to maximize a reward function r will, when deployed, “seek reward” in the sense of adopting r as its effective utility function. This is often *not* the case, especially when the deployment environment differs from the training environment: see Turner’s essay [Reward is not the optimization target](#) (2022) and the goal-misgeneralization experiments of Langosco et al. [43]. A theory of “AI motivations” that could predict an agent’s effective utility function from its training pipeline would be an important advance in AI safety.

generative model: “states z generate observations x .” The belief q is restricted to a simple family $\mathcal{Q}$, because exact Bayes is intractable for large state spaces,¹³ and because a family of representable beliefs is a natural model of a bounded reasoner. The agent tries to make $q \in \mathcal{Q}$ close to the exact posterior $\bar{q}_t(\cdot | x_t)$.

Here “close” is measured by *Kullback–Leibler divergence*. For probability distributions r and s on the same finite set Z ,

$$\text{KL}(r||s) = \sum_{z \in Z} r(z) \log \frac{r(z)}{s(z)}.$$

Gibbs’ inequality says $\text{KL}(r||s) \geq 0$, with equality if and only if $r = s$. In information-theoretic terms, call $-\log s(z)$ the *surprise* of an observer with beliefs s at seeing z ; then $\text{KL}(r||s)$ is the mean excess surprise of an observer who believes the sample was drawn from s when it was drawn from r . It is not symmetric, so it is not a metric; it is a directed measure of how badly s approximates r .

A (base) [language model](#) is a [neural network](#) trained to predict the next [token](#) (word or word-fragment) in a text.¹⁴ Its input is a portion of the text, and its output is a probability distribution for the token that comes next. Multiplying these per-token probabilities yields a probability distribution s over whole texts. The training minimizes an empirical estimate of the average surprise $\mathbb{E}_{x \sim r}[-\log s(x)]$, where r is the true distribution of human writing. This average equals $\text{KL}(r||s)$ plus a constant the model cannot affect, the entropy of human writing itself. To train a language model is to push a KL divergence down.

The active predictor scores each candidate belief $q \in \mathcal{Q}$ against the observation x_t by its *variational free energy*

$$\begin{aligned} F(q, x_t) &:= \mathbb{E}_q[\log q(z) - \log \bar{q}_t(x_t, z)] \\ &= \text{KL}(q || \bar{q}_t(\cdot | x_t)) - \log \bar{q}_t(x_t), \end{aligned} \tag{2}$$

and updates its belief by adopting the $q \in \mathcal{Q}$ that minimizes $F(q, x_t)$. Here $\bar{q}_t(x, z)$ is the predicted joint (1) and $\bar{q}_t(x_t) = \sum_z \bar{q}_t(x_t, z)$ is the probability the agent assigned to the observation x_t before it arrived. Since KL is nonnegative, this identity shows that $F(q, x_t)$ is an upper bound on the agent’s surprise $-\log \bar{q}_t(x_t)$ at seeing x_t . The surprise term does not depend on q . Thus, for fixed x_t , lowering $F(q, x_t)$ lowers this upper bound and makes q a better approximation to the posterior. The first expression for F involves only the unnormalized joint $\bar{q}_t(x_t, z)$ and an expectation under q , never the sum over Z that exact conditioning requires; this is what makes minimizing F over a tractable family cheaper than computing

¹³One cost of exact Bayes is the normalizing sum: computing the posterior $\bar{q}_t(\cdot | x_t)$ requires $\bar{q}_t(x_t) = \sum_z p(x_t | z) \bar{q}_t(z)$, a sum over the whole state space Z . The free energy below is arranged so that this sum is never needed.

¹⁴“Base” refers to the model at the end of this first stage of training, called *pretraining*, on a large corpus of text. The chat assistants served to users are derived from base models by further training to follow instructions and satisfy human preferences [58]. In the past few years, more stages have been added to this pipeline, including *character training* to shape the model’s personality and values [47], and reinforcement learning with verifiable rewards to improve performance on math and coding tasks [42].

the posterior. If $\mathcal{Q}$ contains all distributions on Z , the minimizer is the posterior $\bar{q}_t(\cdot | x_t)$ itself, and the variational update reproduces the Bayes update of View 1; a restricted $\mathcal{Q}$ returns the member nearest the posterior in KL.

Active inference [61] extends the predictor from perception to action. Where View 1 adds a utility, View 2 adds a *goal distribution* g : a probability distribution on X recording which observations the agent would like to receive. A candidate policy π is scored by its *expected free energy*

$$\mathcal{F}(\pi) = \sum_t \mathbb{E}_{x_t \sim Q_\pi} [-\log g(x_t)] - \sum_t I_\pi(z_t; x_t), \quad (3)$$

and the agent selects a policy with a low score. The first term is the expected surprise of the predicted observations, measured against the goal rather than against the prediction itself. The second is the mutual information $I_\pi(z_t; x_t) = \mathbb{E}_{x_t \sim Q_\pi} \text{KL}(Q_\pi(z_t | x_t) \| Q_\pi(z_t))$ between the hidden state and the coming observation under Q_π : the information the agent expects the observation to bring. A creature whose goal distribution puts nearly all its mass on a body temperature of 37°C finds cold surprising, and puts on a coat: a goal is a prediction the agent works to make true.

Two comparisons with View 1 are worth making. Set $u(\tau) = \sum_t \log g(x_t)$; then the first term of $\mathcal{F}(\pi)$ is $-\mathbb{E}_{\tau \sim Q_\pi} [u(\tau)]$, so minimizing it alone is expected-utility maximization for a utility that is additive over time and depends only on observations. Nothing is lost by writing preferences as a distribution: any bounded utility on observations is $\log g$ up to an additive constant. The second term is what View 1 lacks. It is a preference of a second kind, a value placed on what the agent learns rather than on what the world does. A View 1 agent values information only instrumentally, through the planning that exact expected-utility maximization requires; the active predictor values it directly, one step ahead, a cheaper substitute for that planning. If the goal were replaced by the agent’s own prediction, minimizing expected surprise would send it to the most predictable place it can find, a dark and quiet room [24, 52]; the goal distribution and the information term are the standard answers to that objection. The result is a duality between perception and action: perception lowers free energy by changing the belief to fit the world; action, by changing the world to fit the goal.

An AI agent’s world-model is learned during training, not crafted by human engineers. Can we learn the agent’s world-model from its behavior? For a certain class of agents, described in the terms of View 1, the theorem of the next subsection says the answer is yes.

3.4. Inferring an agent’s world-model from its behavior. A *Bayesian network* is a directed acyclic graph G whose vertices are random variables $X_1, \dots, X_n$, together with conditional probability tables satisfying

$$p(x_1, \dots, x_n) = \prod_i p(x_i | \text{pa}(x_i)),$$

where $\text{pa}(x_i)$ denotes the parents of X_i in G . The graph is a bookkeeping device for conditional independence: once the parents of a node are known, its non-descendants add no further information. Pearl’s graphical criterion, *d-separation*, reads off from the graph exactly which conditional independences are forced by this factorization [62].

A *causal* Bayesian network adds an interpretation to the arrows: each conditional distribution is a local mechanism. An *intervention* on a variable replaces its usual mechanism by a chosen value or distribution and leaves the other mechanisms alone. Interventions can distinguish causation from correlation. If smoke and fire are correlated, then observing smoke changes the probability of fire; intervening to make smoke with a smoke machine need not.

The thermostat that began our scale of agency can now be put to the test. Intervene by wiring its output to an air conditioner instead of a heater, and it will chill the room it was built to warm: whenever the temperature drops, the thermostat calls for more, driving it lower still. The thermostat pursues its temperature only while the mechanisms around it stay fixed; by contrast, an agent that kept succeeding across such rewirings would need a world-model to track which mechanism does what. The theorem below makes this necessity precise.

A *causal influence diagram* is a causal Bayesian network with extra decision nodes, where a policy chooses actions from observations, and utility nodes, whose values the agent is scored on. This is a simpler decision setting than the POMDP above: the theorem assumes that the decision is scored directly by the utility nodes and does not change the environment variables that feed them, so there is no chain of consequences to reason through. The agent is tested not just in one environment but across a family of *local interventions*: each replaces the value of some environment variable by a chosen function of it (a constant, or the flip of a binary variable, say), or *masks* some of the agent’s observations, hiding them from view. The agent is told which intervention is in force and responds with a policy; its *regret* under that intervention is the utility gap between its policy and the best policy for that intervened environment.

To illustrate how an agent’s behavior can reveal its world-model if the agent’s utility is known, consider an agent that chooses one of two treatments. A hidden binary variable decides which treatment works, and the agent is scored on curing the patient. Externally set the hidden variable to 1 with probability α —an intervention—and watch the agent as α varies. An agent playing optimally switches treatments at the threshold α^* where the two treatments’ expected utilities cross, and that indifference equation can be solved for the agent’s implicit estimates of the treatments’ effects. The next theorem turns this trick into a general technique for extracting an agent’s world-model from its behavior.

Theorem (Robust agents contain causal world models; Richens and Everitt [65]). *For almost all causal influence diagrams satisfying their regularity assumptions, any agent that responds to each local intervention, maskings included, with a policy of regret at most δ determines an approximate causal model of the utility-relevant environment, with error bounded by a function $\gamma(\delta)$ satisfying $\gamma(0) = 0$ and growing*

linearly for small δ . In the case $\delta = 0$, the causal graph and the joint distribution over all ancestors of the utility are identified exactly.

The proof is constructive: it is the switch-point trick above, repeated. Treating the policy as an oracle, mix interventions pairwise by a parameter α and watch where the optimal action switches; because the utility is known, each indifference equation solves for one interventional probability, and together they recover the conditional distributions and parent sets, except on measure-zero degeneracies such as exact cancellations or ties. The theorem lives in View 1: the utility is given, and what behavior reveals is the model. What is extracted is a causal model of the environment implied by the agent’s competent responses; the theorem says nothing about how, or whether, that model is represented inside the agent. Recovering goal and model together from behavior alone is harder, and without further assumptions it is not possible even in principle: the same behavior is consistent with many pairings of a model with a goal [6].

In a follow-up, Richens, Everitt, and Abel [66] prove a companion result for goal-directed agents: any agent that generalizes across a large enough family of multi-step goals must contain an accurate predictive model of its environment, in the sense that its policy alone determines the environment’s transition probabilities, and an algorithm extracts them from the policy. The more capable the agent, the more accurate the extracted model: for goals that chain n sub-goals in sequence, the error in each extracted transition probability is $O(1/\sqrt{n})$, even for agents that usually fail.

Extraction raises a question of independent interest: the recovered model comes expressed in *some* set of latent variables—whose? If two models predict identically, must their internal variables be translatable into one another, or could an alien mind carve the world into concepts that ours cannot express? Wentworth and Lorell’s *natural latents* are a first answer: conditions, robust to approximation, under which the latent variables of two predictively equivalent models must translate into one another’s [78]; Eisenstat’s *condensation* [22] approaches the same question from probability, asking how an agent’s concepts condense out of its predictive state.

★ **Open Problem MAIS-02 (Recovering world-models from behavior).** Fix a finite causal influence diagram with binary variables and known utility, and an agent whose policy has regret at most δ across the family of local interventions. The extraction algorithm behind the theorem above [65] recovers the agent’s causal model, to within $\gamma(\delta)$, from unlimited exact queries to such a policy. Prove a finite-sample version: if each query returns an action sampled from the policy, rather than the policy’s exact action probabilities, and each experiment draws on the same intervention family as the theorem, maskings included, how many experiments determine the graph and the conditional probability tables to within η , and how does the answer scale with δ ? As a first case, take the two-variable diagrams, where the *identified set*—the set of models consistent with the agent’s observed behavior—can be computed exactly, and measure how the extraction algorithm degrades when its indifference equations are estimated from samples.

Interventions on the agent’s inputs, or on the [activations](#) (the values computed inside its network), are the harder surfaces beyond.

4. ALGEBRA: WHAT IS A LEARNED FEATURE?

Interpretability (Section 1.3) tries to reverse-engineer a trained neural network into something a human can read. Much of its core is linear algebra and representation theory: how a network stores more concepts than it has dimensions, and what it means for a concept to be a *direction* one can probe or steer.

4.1. Superposition: packing features into directions. How does a neural network with only thousands of dimensions per layer represent the millions of distinct concepts (“[features](#)”) it seems to know? At a high level, a [transformer](#) language model maps a sequence of tokens to a sequence of next-token predictions, one for each position:

token sequence $\mapsto$ sequence of vectors in $\mathbb{R}^n \mapsto$ next-token predictions.

The copy of $\mathbb{R}^n$ in which these intermediate vectors live is the network’s [activation space](#). Each of the two maps is a composition of [layers](#). A layer is a map $x \mapsto \sigma(Wx+b)$: an affine map, whose matrix and vector entries are the network’s [weights](#), followed by a fixed nonlinear function $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ applied to each coordinate. A [neuron](#) is one coordinate of a layer. (A transformer alternates such layers with attention layers, which mix information across positions.)

The [superposition](#) hypothesis of Elhage et al. [23] says that when features are sparse, a network can store more than n of them, as directions in activation space that are *not* orthogonal but only nearly so, tolerating the resulting interference because at any moment only a few features are active (Figure 3). In a small [ReLU](#) model one can watch the geometry organize itself into regular polytopes—antipodal pairs, triangles, pentagons—as a function of how sparse and how important the features are. This toy network compresses $x \in [0, 1]^m$ to $Wx \in \mathbb{R}^n$ and decompresses with W^T (a *tied-weight autoencoder*). The coordinates of x are independent and each is zero with high probability, and $I_i > 0$ measures the importance of feature i . The training problem, over $W \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^m$, is

$$\min_{W,b} \mathbb{E}_x \left[\sum_{i=1}^m I_i (x_i - \text{ReLU}(W^T W x + b)_i)^2 \right].$$

The columns of W are the feature directions in $\mathbb{R}^n$. This minimization problem resembles the Thomson problem of arranging m unit charges on the sphere S^{n-1} to minimize the Coulomb energy.¹⁵ Regular polytopes appear in the solutions to both problems.

The nearest classical analogue is *compressed sensing* [10, 19]. Picture the features active on a given input as a sparse vector $x \in \mathbb{R}^m$ (only a few of the m possible features fire at once); the network stores it as $y = \Phi x$ in the $n \ll m$ dimensions

¹⁵A special case reminiscent of the Thomson problem is: one-sparse x , equal importances, zero bias, unit-length columns, so that the problem becomes to minimize $\sum_{i \neq j} \text{ReLU}(W_i \cdot W_j)^2$ over unit vectors $W_1, \dots, W_m \in S^{n-1}$, where W_i is the i th column of W .

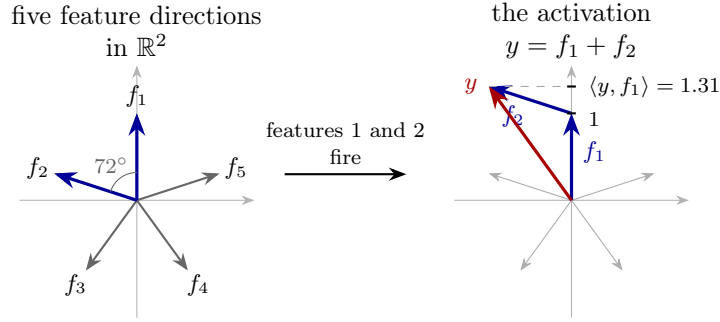


FIGURE 3. Superposition illustrated by five features in $\mathbb{R}^2$, their directions forming a regular pentagon, so no two are orthogonal. When features 1 and 2 fire, the activation is $y = f_1 + f_2$. Projecting y onto f_1 returns 1.31 rather than 1; the excess is the interference from f_2 , modest because adjacent directions are 72° apart, and tolerable as long as few features fire at once.

of its activation space—a linear *measurement*—and reading a feature back out is the *recovery* of x from y . Two classical facts explain how this can work. First, there is room for the directions: for every $\varepsilon \in (0, 1)$, the space $\mathbb{R}^n$ contains $e^{\Omega(\varepsilon^2 n)}$ unit vectors whose pairwise inner products are at most ε in absolute value, so the number of ε -almost-orthogonal directions grows *exponentially* in the dimension (a consequence of the Johnson–Lindenstrauss lemma [37]). And second, when only a few of those directions are active at once, the stored signal can be read back out. Say that $\Phi \in \mathbb{R}^{n \times m}$ is *restricted-isometric of order k* with distortion δ if

$$(1 - \delta)\|x\|^2 \leq \|\Phi x\|^2 \leq (1 + \delta)\|x\|^2$$

for every k -sparse vector x , meaning one with at most k nonzero entries.

Theorem (Candès [11]; sharp constant, Cai–Zhang [9]). *If Φ is restricted-isometric of order $2s$ with distortion $\delta < 1/\sqrt{2}$, then every s -sparse $x \in \mathbb{R}^m$ is the unique minimizer of*

$$\min_z \|z\|_1 \quad \text{subject to} \quad \Phi z = \Phi x,$$

and so is recovered exactly by a convex program. A random Φ , say with independent Gaussian entries suitably normalized, has the property with high probability once

$$n \gtrsim s \log(m/s).$$

The significance of the $s \log(m/s)$ scaling is that it allows for exact recovery when the dimension n is only logarithmic in the total number of features m (and linear in the number of features s active *at once*).

There is a catch. Compressed sensing assumes the dictionary Φ is *known*. What if someone hands you a trained network and asks what features it represents? You can run inputs through the network and record activation vectors, but the feature directions were not supplied with the weights (the network’s trained parameters). Recovering the features is therefore a problem in *dictionary learning*: from the

observed activation vectors alone, simultaneously infer a dictionary Φ whose columns are feature directions and, for each activation y , a sparse coefficient vector x with $y \approx \Phi x$. Both the dictionary Φ and the codes x are unknown.

A [sparse autoencoder](#) attempts this empirically. It is trained to reconstruct activation vectors using a learned dictionary, with an ℓ^1 penalty encouraging each reconstruction to use only a few dictionary directions. The resulting directions are often more interpretable than the network’s raw neurons [8, 17]. But does the sparse autoencoder recover features actually used by the network, or does it impose a new coordinate system that merely makes the activations easier to describe? Classical dictionary learning answers this question when the codes are exactly sparse and their supports well spread: the dictionary is then determined up to relabeling and rescaling of its columns [2, 33], stably in the presence of noise [29, 25]. Network activations flout these hypotheses because their features co-occur, which is where the first open problem below begins.

4.2. Probing, steering, and learned representations. What could it mean for a concept to be a direction in activation space? Consider incomplete sentences such as “The keys on the table . . .” and pause the computation at the activation h_T of the last token. A linear functional ℓ [probes](#) grammatical number if, for some threshold c , the inequality $\ell(h_T) > c$ predicts that the subject is plural. A vector $v \in \mathbb{R}^n$ *steers* toward the plural if replacing h_T by $h_T + v$ raises the log-probability of “are” relative to “is.” Thus the same example produces a probing covector ℓ and a steering vector v . Relating them requires an inner product.

Park, Choe, and Veitch [60] formalize these two experiments for next-token prediction. They ask whether activation space carries a *causal inner product* in which independently varying concepts are orthogonal and whose Riesz map sends probing covectors to steering vectors. They estimate this geometry from the unembedding matrix (the model’s final linear map, from activation space to next-token scores).

Arditi et al. [5] show that in several publicly released chat models, refusal to answer the user’s query is mediated by a single direction in the [residual stream](#) (the running activation vector that a transformer’s layers read from and write to): this direction serves both as a probe for refusal (it predicts whether the model will refuse) and as a steering vector (adding or subtracting it turns refusal on or off).

Refusal is a semantic behavior, with no single ground truth for how the network *should* behave. Algebra is a cleaner laboratory: train a neural network on part of the multiplication table of a finite group, then test whether it generalizes correctly and what algorithm it learns. Representation theory appears in trained circuits of this type. A one-layer transformer trained to add integers modulo p learns a discrete-Fourier “clock” algorithm, with the characters of $\mathbb{Z}/p\mathbb{Z}$ appearing in its weights [54]. Chughtai, Chan, and Nanda [13] take the first step beyond this abelian case. The ambient structure is the Artin–Wedderburn decomposition of the group algebra into matrix blocks,

$$\mathbb{C}[G] \cong \bigoplus_{\rho} M_{d_{\rho}}(\mathbb{C}), \quad \sum_{\rho} d_{\rho}^2 = |G|,$$

one block per irreducible representation ρ of G , where d_ρ is the dimension of ρ . Trained neural networks compute the product using a subset of the irreducible representations. The correctness of this algorithm is a representation-theory theorem; the fact that trained networks implement it is an empirical finding. *Which* irreducible representations training selects is still something we discover after the fact.

Sometimes this structure is forced by a theorem rather than found by observation. Marchetti et al. [48] consider networks whose inputs are functions on a finite group G and whose weights are pinned down by the function the network computes, up to a unitary change of basis at each neuron. They prove that if the network is invariant under translation of its input by G , then the weights of each neuron (matrix-valued, if G is nonabelian) form an irreducible unitary representation of G , up to a fixed linear map. If the weights are moreover orthonormal, then together they form the Fourier transform on G , and the multiplication table of G can be read off the weights.

What makes these circuits a safety topic is *selection*: several different algorithms fit the training data equally well, and training picks one of them without telling us which. In the group example, the choice is which irreducible representations to use. Two networks with equally low training [loss](#) can behave differently on inputs unlike any they were trained on, and the difference is decided by which algorithm training found. To predict behavior on such inputs, we need to know the algorithm, not only the loss.

One caveat carries across this whole enterprise: a probe that *predicts* a feature may only be correlated with it. Showing that the network actually *uses* the direction requires an intervention such as *activation patching*: overwrite the component of the activation with the value it takes on a different input, and measure whether the behavior changes.

4.3. Open problems. A catalogue of open problems in interpretability is collected by Sharkey et al. [71]. The problems below sharpen two of theirs and add a third.

★ **Open Problem MAIS-03 (The geometry and identifiability of superposition).** Suppose activations have the form $y = \Phi x + \xi$, where the columns $v_1, \dots, v_m$ of Φ are unit feature directions, ξ is noise, and the sparse codes x have correlated supports. Estimate the dictionary the way a sparse autoencoder does: minimize reconstruction error plus an ℓ^1 penalty on the codes, over dictionaries whose columns have unit norm. Determine, in terms of the coherence $\mu = \max_{i \neq j} |\langle v_i, v_j \rangle|$, the sparsity, the sample size, and the support correlations, when every minimizer recovers the true directions up to permutation and scaling, and when it instead *merges* co-occurring directions into one.

★ **Open Problem MAIS-04 (Training for interpretability).** Post-hoc interpretability asks whether a trained network’s features can be recovered after the fact. A complementary problem is to train networks whose features are easier to recover in the first place. In the ReLU toy model of Elhage et al. [23] the features

are known by construction, so the trade-off can be made exact. Determine the interference–performance frontier: for features appearing sparsely and independently, among weight matrices whose coherence (the largest $|\langle W_i, W_j \rangle|$ over pairs $i \neq j$ of columns, after normalizing the columns to unit length) is at most μ , how small can the task loss be, as a function of μ , the sparsity, the number of features, and the number of neurons? A first case: prove or refute that penalizing the *average* of $\langle W_i, W_j \rangle^2$ over pairs during training lowers the coherence (the *maximum* over pairs) of the minimizer, compared with training on the task loss alone.

★ **Open Problem MAIS-05 (Representation theory of learned circuits).**

Networks trained on group multiplication learn representation-theoretic algorithms [54, 13], but *which* irreducible representations (irreps) they use varies from one random seed to the next. Fix a finite group G , a one-hidden-layer architecture of fixed width, Gaussian initialization, and gradient flow on the cross-entropy loss (the mean negative log-probability assigned to the correct product) with *weight decay*, an added penalty proportional to the squared norm of the weights. The irreps visible in the trained network’s outputs then form a random subset of the irreps of G ; determine its distribution—which irreps are learned, with what probability, as a function of the width and the decay strength. The tables of learned representations in [13] are the data such a theorem would have to explain.

5. ANALYSIS AND GEOMETRY: WHICH SOLUTION DOES TRAINING FIND?

5.1. A Bayesian account of generalization: singular learning theory.

A trained network is a setting of parameters chosen to fit its *training data*, the finite set of examples it was optimized on. The trouble is that a large network has astronomically many settings that fit those examples perfectly, and they disagree wildly everywhere else. The optimizer, [gradient descent](#), which repeatedly nudges the parameters downhill on the training error, must land on one of them. For safety, the selected solution matters: a network that behaved well while we were watching may behave differently once deployed. So why do the solutions found by training *generalize*, that is, predict well on fresh data from the same source, when so many other data-fitting solutions would not? Large networks reach zero error even when the labels are replaced by pure noise, and a two-layer network with only $2n + d$ parameters can fit *any* labeling of n points in $\mathbb{R}^d$ (Zhang et al. [80]); the classical bounds that explain generalization by counting parameters say nothing here.

Singular learning theory [76] gives a precise answer for Bayesian prediction in singular statistical models; whether its geometric account also predicts the solutions selected by gradient descent is a central open question, taken up in the problem *Opposing staircases* below. Realistic models are *singular*: their Fisher information degenerates somewhere. The usual reason is that distinct parameters can compute the same function, so the optimal set is not a single point but a positive-dimensional real-analytic variety, along which the loss is constant and its Hessian therefore degenerate (Figure 4). The classical large-sample asymptotics all assume an isolated optimum with nondegenerate Hessian, the bowl of the left panel; the right replacements come from algebraic geometry.

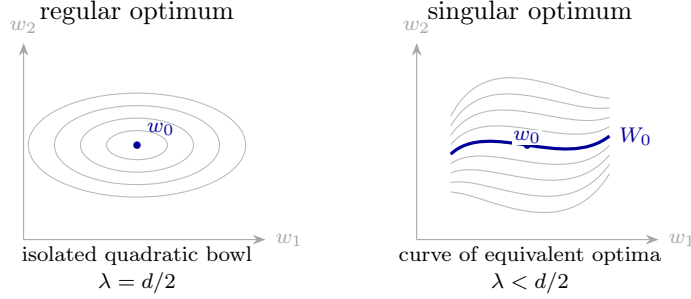


FIGURE 4. Regular asymptotics treat the optimum as an isolated quadratic bowl, giving the classical $\frac{d}{2} \log n$ complexity penalty. Singular learning theory allows a whole curve or surface of parameters to compute the same function: in the right panel the optimal set is a curve W_0 through w_0 , and the learning coefficient λ , which replaces $\frac{d}{2}$ in the complexity penalty, satisfies $\lambda < \frac{d}{2}$, as it does whenever the optimal set has positive dimension.

As a simple example, take first the regular loss $K(w) = w^2$ on the interval $[-1, 1]$: the set where $K \leq \varepsilon$ is an interval of length $2\sqrt{\varepsilon}$, so the volume of near-optimal parameters scales as $\varepsilon^{1/2}$, that is, as $\varepsilon^{d/2}$ with $d = 1$. Now take $K(a, b) = a^2 b^2$ on the square $[-1, 1]^2$. Its zero set is not a point but the union of the two axes, crossing at a singularity, and the region where $K \leq \varepsilon$ is a neighborhood of the axes of area of order $\varepsilon^{1/2} \log(1/\varepsilon)$: the exponent stays $1/2$ rather than rising to $d/2 = 1$, and a logarithm appears. A singular loss has far more near-optimal parameters than its dimension suggests, and the pair (exponent, power of the logarithm) is the simplest instance of the learning coefficient and multiplicity about to be defined.

To make this precise, fix a model $p(x | w)$ with parameter w in a compact $W \subseteq \mathbb{R}^d$, a prior φ on W (a probability density, positive on W , encoding which parameters are plausible before any data is seen), and a *true distribution* q_0 , the one the data will actually be drawn from, assumed to be one the model can represent exactly: $q_0(x) = p(x | w_0)$ for some $w_0 \in W$. Write

$$K(w) = \text{KL}(q_0 \| p(\cdot | w)) = \int q_0(x) \log \frac{q_0(x)}{p(x | w)} dx,$$

so that the set of *optimal parameters* $W_0 = K^{-1}(0)$ is exactly where the model recovers q_0 .

Given independent samples $X_1, \dots, X_n$ from q_0 , the *Bayes free energy* is defined by

$$\bar{F}_n = -\log \int_W \prod_{i=1}^n p(X_i | w) \varphi(w) dw.$$

The integral is the probability the model assigns to the sample before any parameter is chosen: the probability $\prod_i p(X_i | w)$ that parameter w assigns, averaged over the prior. So $\bar{F}_n$ is small when the model, prior included, predicted the data well; and it

is random, since it depends on the sample. (This is not the variational free energy $F(q, x)$ of Section 3, which scores a belief against a single observation: that one bounds a negative log evidence from above, and this one is a negative log evidence.)

Take first the *regular* case, which imposes two conditions. The model is *identifiable*: distinct parameters give distinct distributions, so the optimal set is a single point $W_0 = \{w_0\}$. And the Hessian of K at w_0 , which is the *Fisher information* $I(w_0)$, is positive-definite. Under both conditions the integrand defining $\bar{F}_n$ concentrates in a shrinking neighborhood of w_0 ; expanding $\log p(X_i | w)$ to second order there makes the integral Gaussian, contributing a volume factor $(2\pi/n)^{d/2} \det I(w_0)^{-1/2}$. Taking $-\log$,

$$\bar{F}_n = nS_n + \frac{d}{2} \log n + O_p(1), \quad S_n = -\frac{1}{n} \sum_i \log q_0(X_i),$$

where S_n is the empirical entropy and the remainder is bounded in probability.¹⁶ The first term is the score the true distribution itself would earn on the sample, and no model can beat it on average. The second is the price of not knowing w_0 : the posterior concentrates in a ball of radius of order $n^{-1/2}$ about w_0 , whose prior volume is of order $n^{-d/2}$, and minus the log of that volume is $\frac{d}{2} \log n$. Every parameter costs half a log n , so this term is the *complexity penalty*: what a model with more parameters pays, when models are compared by free energy, for its extra freedom to fit. Watanabe's theorem is the singular replacement, in which $d/2$ gives way to an invariant of the singular geometry.

That invariant is defined by volume growth. Write $v(\varepsilon) = \int_{\{K \leq \varepsilon\}} \varphi(w) dw$ for the prior volume of the parameters fitting the truth to within ε . For real-analytic K and a smooth positive prior, this volume has the asymptotic form

$$v(\varepsilon) \sim c \varepsilon^\lambda (\log(1/\varepsilon))^{m-1} \quad (\varepsilon \rightarrow 0)$$

for a constant $c > 0$, a rational number $\lambda > 0$, and an integer $m \in \{1, \dots, d\}$. The exponent λ is the *learning coefficient* of the model and m is its *multiplicity*. In the regular case the set $\{K \leq \varepsilon\}$ is an ellipsoid of radius of order $\sqrt{\varepsilon}$, so $\lambda = d/2$ and $m = 1$; in the toy loss $a^2 b^2$ above, $\lambda = 1/2$ and $m = 2$. Smaller λ means near-optimal parameters are more plentiful.

Theorem (Watanabe's free-energy asymptotics [76]). *For a singular model satisfying Watanabe's regularity conditions (real-analyticity and a mild variance bound¹⁷), as $n \rightarrow \infty$,*

$$\bar{F}_n = nS_n + \lambda \log n - (m-1) \log \log n + O_p(1), \quad (4)$$

where λ and m are the learning coefficient and multiplicity of the model. They take the places of the $d/2$ and the 1 of the regular case. The $O_p(1)$ remainder

¹⁶ $O_p(1)$ means *bounded in probability*: for every $\varepsilon > 0$ there is an M with $\Pr(|\cdot| > M) < \varepsilon$ for all large n .

¹⁷The *relatively finite variance* condition: writing $f(x, w) = \log(q_0(x)/p(x | w))$, $K(w) = \mathbb{E}_{q_0} f(X, w)$, and $V(w) = \mathbb{E}_{q_0} f(X, w)^2$, one assumes $V(w) \leq C K(w)$ near the optimal set: the variance of the log-likelihood ratio is controlled by its mean.

converges in distribution.¹⁸ The same coefficient controls generalization. Let $\hat{p}_n$ be the average of $p(\cdot | w)$ over the posterior, the probability density on W proportional to $\prod_i p(X_i | w) \varphi(w)$ (the prior reweighted by how well each parameter fits the data). Then the generalization error $G_n = \text{KL}(q_0 \| \hat{p}_n)$ obeys $\mathbb{E}[G_n] = \lambda/n + o(1/n)$.

In the regular case the theorem recovers the expansion above; in a singular model $\lambda \leq d/2$, often strictly. The expansion localizes: restrict the integral defining $\bar{F}_n$ to a neighborhood of one optimal parameter, and the expansion (4) holds with that neighborhood’s own coefficient. Among the optimal parameters (all fitting the truth exactly, all weighted by the prior), the posterior share of the most degenerate neighborhoods, those of smallest local λ , grows with n . In this Bayesian setting the posterior concentrates on the solutions that the formula $\mathbb{E}[G_n] = \lambda/n$ marks as best-generalizing. This is the theory’s rendering of the slogan that learning prefers “simpler” solutions: simplicity means a smaller learning coefficient λ , not fewer parameters.

The degeneracy behind a small λ is largely *structural*: it comes from directions in parameter space along which the model $p(\cdot | w)$, and hence the loss, does not change, and these are what pull λ below $d/2$. For instance, if a column of one weight matrix is zero, the corresponding row of the matrix before it can be changed freely.

The theory is exact for *Bayesian* learning, not directly for stochastic gradient descent (gradient descent driven by noisy, small-batch estimates of the gradient). Bridging the two is the problem *Opposing staircases* below. The two are closer than they appear: Khan and Rue [38] derive stochastic gradient descent, along with many other optimizers, as approximations of a single Bayesian update rule.

There are two ways to compute λ . The first is by hand: algebraic geometers know λ as the *real log canonical threshold* of K relative to φ , and one computes it by resolving the singularities of K (Hironaka [34]), which is feasible for small models. The second is by sampling. Introduce a knob $\beta > 0$ and the *tempered posterior*

$$p_\beta(w) \propto \varphi(w) e^{-\beta n L_n(w)}, \quad L_n(w) = -\frac{1}{n} \sum_i \log p(X_i | w),$$

which interpolates between the prior ($\beta = 0$) and concentration on the best-fitting parameters ($\beta \rightarrow \infty$). At $\beta = 1/\log n$, the average of nL_n over the tempered posterior exceeds its minimum by $\lambda \log n$, to leading order [77]. So λ can be *estimated* by sampling the tempered posterior, needing neither the true distribution nor an explicit resolution of singularities. (In statistics, taking $\beta < 1$ hedges against a misspecified model: Grünwald’s “Safe Bayesian” learns β from the data [30], and Alquier and Ridgway prove that tempered posteriors still concentrate [4].)

Sampling a density in millions of dimensions is not a small matter, but the diffusion $dw_t = -\nabla U(w_t) dt + \sqrt{2} dB_t$ with $U = \beta n L_n - \log \varphi$ has stationary

¹⁸In the regular case the limit is $C - \frac{1}{2}\chi_d^2$ for a constant C , the Bayesian shadow of Wilks’ theorem, since the log-likelihood-ratio statistic $2(\sup_w \sum_i \log p(X_i | w) - \sum_i \log p(X_i | w_0)) \Rightarrow \chi_d^2$. In a singular model the limit is in general non-Gaussian, a functional of a Gaussian process on the manifold obtained by resolving the singularities of K .

density proportional to e^{-U} , which is p_β ; run in discrete time with small-batch gradients, it is gradient descent with Gaussian noise added at each step, and this is the sampler used in practice [44]. That practicality is what lets the theory meet networks far too large to analyze by hand.

What one estimates is not the global λ but the *local learning coefficient*: the same invariant computed in a neighborhood of the particular solution w^* that training actually reached, measuring how degenerate *that* solution is [44]. As a network trains, its estimated local learning coefficient is observed to jump at discrete moments, called *phase transitions*: the solution moves to a qualitatively new, differently-degenerate part of the landscape as a new piece of internal structure forms. Tracking these jumps is the program of *developmental interpretability*: so far on relatively small networks and with preliminary validation, though related estimators from the same theory have already surfaced tens of thousands of candidate structures inside a billion-parameter language model [53].

Two networks can fit the same training data by different internal mechanisms, and two mechanisms that agree on the training distribution can disagree on inputs beyond it. A held-out test drawn from that distribution cannot tell them apart. In the Bayesian setting the geometry decides which mechanism is selected: the most degenerate solution, the one of smallest λ , receives the most posterior weight. Whether gradient descent selects the same way is the bridge problem *Opposing staircases* below. Nothing in the theory says that the selected solution implements the behavior intended rather than a shortcut that mimics it on the training distribution and fails beyond. Whether small λ favors mechanisms that are also interpretable and robust is the question this program must answer for safety; the case that safety turns on such questions is made in [63].

★ **Open Problem MAIS-06 (Does geometric simplicity force a legible mechanism?)** Fix an odd integer p and $H \geq 2p - 1$. On input $(a, b) \in (\mathbb{Z}/p\mathbb{Z})^2$, consider the one-hidden-layer network

$$f_w(a, b) = \sum_{j=1}^H V_j (u'_j(a) + u''_j(b))^2 \in \mathbb{R}^p,$$

trained, with squared-error loss averaged over all p^2 inputs, to output the coordinate vector indexed by $a + b$. The j th summand is one hidden neuron: it adds two entries from the lookup tables $u'_j, u''_j: \mathbb{Z}/p\mathbb{Z} \rightarrow \mathbb{R}$, squares the result, and writes the vector V_j to the output. Fourier inversion gives exact fits in which each neuron that contributes nontrivially carries a single frequency: for some k , both lookup tables are a constant plus a linear combination of $\cos(2\pi k \cdot /p)$ and $\sin(2\pi k \cdot /p)$. Up to a change of basis, this is the algorithm found by reverse-engineering trained networks [54].

Is this Fourier structure forced by singular geometry? In a fixed closed ball containing one of these Fourier fits, prove or refute that every exact fit of smallest local learning coefficient has this single-frequency form, apart from neurons whose contribution vanishes identically. The first non-vacuous case is $p = 5$ with nine

hidden neurons: use its Fourier symmetries to classify the most singular exact fits, or find a non-Fourier one by computer algebra. A proof would connect geometric simplicity to a mechanism one can read from the weights; a counterexample would show that the two notions of simplicity can part ways.

5.2. Learning in stages. Learning does not always proceed smoothly; long flat stretches give way to sudden changes.

A solvable model of such stepwise learning is the deep *linear* network: a product of matrix factors $W_L \cdots W_1$ fit to input–output pairs (x, y) by *gradient flow* (gradient descent in continuous time) on the squared error $\frac{1}{2}\mathbb{E}\|y - W_L \cdots W_1 x\|^2$. Assume the inputs are *whitened* (linearly transformed so that $\mathbb{E}[xx^\top] = I$) and write $\Sigma = \mathbb{E}[yx^\top]$ for the input–output correlation matrix. The system is then

$$\tau \dot{W}_\ell = (W_L \cdots W_{\ell+1})^\top (\Sigma - W_L \cdots W_1) (W_{\ell-1} \cdots W_1)^\top, \quad \ell = 1, \dots, L,$$

with τ a time constant and empty products equal to the identity. The composite map is linear, but as a function of the separate factors the loss is a non-convex polynomial, so the flow is nonlinear. Saxe, McClelland, and Ganguli [69] show that, for initial weights aligned with the singular vectors of Σ , the singular value decomposition *decouples* the dynamics into independent scalar equations, one per singular value s : for two factors started in balance (equal coefficients in the two factors), each is the logistic equation $\tau \dot{a} = 2a(s - a)$, whose solution rises sigmoidally from near 0 to s over a window of width of order τ/s , after a delay that grows only logarithmically as the initial value shrinks (Figure 5). The decoupling and the scalar solutions are exact; the initialization then sets the order of events: starting from small initial weights, modes with larger s turn on earlier, and when their turn-on times are well separated the training error falls not smoothly but in a staircase: long plateaus near saddle points of the loss, each broken by a sharp drop as the next mode is acquired. Here “which structure the network learns, and in what order” has a closed-form answer.

The modular-addition task above shows a cousin of the staircase, not the same phenomenon: a network fits its training data early but generalizes only much later, and abruptly. This is *grokking* [54], a delayed jump in performance on *held-out* data after the training loss is already low, whereas the staircase is a stepwise fall of the *training* loss itself. Both are read, in this program, as transitions between differently-singular regions of parameter space.

The staircase survives beyond the linear case: Abbe, Boix-Adserà, and Misiakiewicz [1] prove that training a two-layer network on a *sparse* target (one depending on only a few of its inputs) again proceeds in stages, saddle to saddle, with the waiting time before each stage set by a combinatorial *leap complexity* measuring how many inputs must be assembled at once. Carrying such exact accounts to the nonlinear networks used in practice is open.

★ **Open Problem MAIS-07 (Opposing staircases).** Watanabe’s theorem is Bayesian; gradient descent is not. (What bridging the two would mean for safety is discussed in [63].) Build the bridge first in the deep-linear staircase above, where

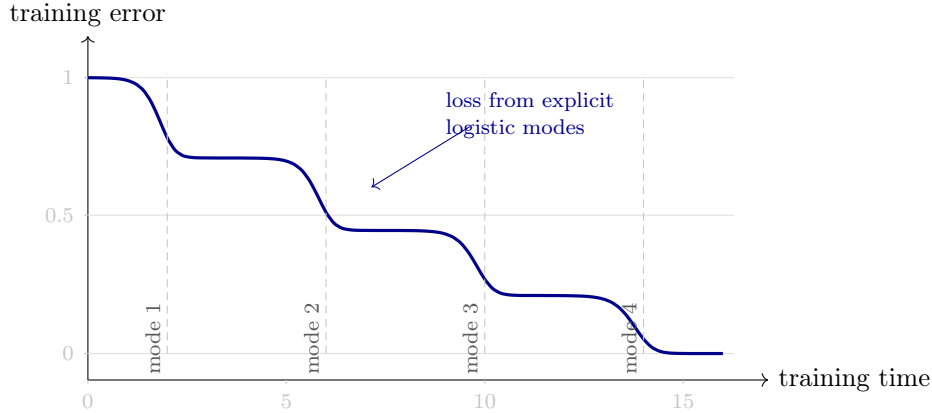


FIGURE 5. A learning staircase computed from the deep-linear solution of Saxe et al. In mode i , s_i is the target singular value and $a_i(t)$ is the coefficient learned by time t , satisfying $\tau \dot{a}_i = 2a_i(s_i - a_i)$. The curve plots normalized training error $\sum_i (s_i - a_i(t))^2$ for $\tau = 0.5$ and $s = (1, 0.95, 0.9, 0.85)$, with small initial coefficients chosen so the four modes turn on at separated times. Each mode turning on produces one drop.

both sides are explicit. Each plateau sits at a saddle where only the k largest modes have been learned, and there the definition itself is part of the problem: the local learning coefficient is defined at a local *minimum*, as the volume exponent of the set of nearby parameters fitting almost as well, but a saddle has descent directions, so that volume does not shrink to zero. Formulate the right local invariant at the k th saddle (one candidate, the *two-sided* exponent: the volume of nearby parameters whose loss lies within ε of the saddle's, above or below, shrinks like ε^λ), compute it for the deep-linear network, and prove or refute the monotone picture that gives this problem its name: as gradient flow descends the staircase of losses, the invariant that, on the Bayesian side, controls generalization climbs an opposing staircase. As a first check, the estimator of Lau et al. [44] runs at any parameter, saddle or not; run it along a simulated staircase and see.

5.3. Generalization beyond the training distribution. The theory above concerns generalization *in distribution*: test data drawn from the same law as the training data. The safety-critical regime is the opposite one, *out of distribution*: how a trained system behaves on inputs unlike anything it saw in training. Two parameter settings with identical training loss can implement different mechanisms (as above) and therefore extrapolate differently; which extrapolation gradient descent selects is, at present, something we observe after the fact rather than predict.

In *goal misgeneralization* [43], a system under *distribution shift* (a deployment input distribution unlike the one it trained on) keeps its *capabilities* while pursuing a *different goal* from the one intended. The standard example is an agent trained to collect a coin in a video game where the coin always sat at the right end of the

level: it learned “move right,” not “reach the coin,” and the two came apart the moment the coin moved.

★ **Open Problem MAIS-08 (A predictive theory of out-of-distribution generalization).** In the coin-collecting environment of Langosco et al. [43], reduced to one dimension, two policies fit the training data perfectly: “move right” (the proxy) and “go to the coin” (the intended goal), which agree whenever the coin sits at the right end. Train a two-layer network by gradient descent on the logistic loss to imitate optimal play, randomizing the coin’s position in an ε -fraction of training episodes. Determine the probability, over the random initialization, that the trained network follows the proxy on a probe state where the two policies disagree, as a function of ε , the width, and the input encoding. For linear policies the answer is a theorem—the encoding, not the initialization, decides—and the standard infinite-width limits [36, 50, 12] follow it under their convergence hypotheses; for networks of finite width it is open at every ε , including $\varepsilon = 0$, already at width two.

6. GETTING STARTED IN AI SAFETY

Technological progress is a human choice, not a law of nature. Mathematics holds truth-seeking as a core value, but the civic mathematician also asks what our truths are for, and whether they can help design AI that is more legible, steerable, and cooperative with us.

The open problems in this invitation are meant as entry points into the new field of Math for AI Safety. Many more problems can be found in the [MAIS repository](#), and the [AI Safety Formalization Atlas](#), maintained by Mario Brčić, collects machine-checked Lean proofs of results relevant to AI safety, including verdicts on submitted solutions to MAIS problems. I hope you’ll pick a problem whose language already feels like home, strip it down to the simplest interesting case, and start proving things!

The rest of this section collects useful links to the growing ecosystem of AI safety organizations and funders.

Upskilling. The [ARENA](#) curriculum teaches the engineering side; the month-long [Iliad Intensive](#) is a full-time course on the theory, aimed at mathematicians, physicists, and theoretical computer scientists. [MATS](#) and [SPAR](#) pair newcomers with mentors for a first research project.

Institutes. Research organizations in AI safety include [ARC](#), [CHAI](#), [Constellation](#), [FAR AI](#), [MAISI](#), [RESI](#), and [Resolution](#). Many run long-term fellowship or visitor programs.

Meetings. [ILIAD](#) is a conference on mathematical approaches to AI alignment, and FAR AI runs a series of [Alignment Workshops](#).

Funding. Grantmakers supporting work in AI safety include [Coefficient Giving](#), [CAIF](#), [SFF](#), and [UK AISI](#).

ACKNOWLEDGMENTS

I thank Jesse Hoogland, Hyojeong Son, Jacob Tsimmerman, Claude, and Codex for many inspiring conversations. Scott Aaronson, Ahmed Bou-Rabee, Vince Conitzer, Jonathan Gabor, Chris Hillar, Brad Knox, Phil Sosoe, Samuel Speas, Kate Stange, Steve Strogatz, Ariel Yadin, and an anonymous referee provided valuable feedback on an early draft.

AI COLLABORATORS

I wrote the first draft in collaboration with Claude Opus 4. To start, I instructed Opus to read the full text of approximately 100 AI safety papers and produce a summary of the main results and techniques of each, along with possible relevance to this invitation paper. I supplied the overarching structure of the invitation (organized by field, ending each section with an open problem) and wrote parts of the introduction. Opus then used its summaries along with samples of my writing to write a draft of each section in my voice, which I edited extensively. Later revisions were polished with the help of Claude Fable 5, GPT 5.6 Sol, and GPT 6 Astra.

In July 2026 an exuberant Fable produced approximately 200 pages worth of open problems in a single night while I was asleep. I asked Sol to audit these for openness, well-posedness, and plausible AI safety relevance. The problems that passed this audit were used to seed the [master list of open problems](#) in the [Math for AI Safety \(MAIS\) repository](#), a new hub for open-source collaboration.

The audit also turned up a small surprise: while checking the problem that became MAIS-01, Sol found a gap in the published proof of Critch’s bounded Löb theorem, and proposed a repair ([MAIS-P2](#)). Critch corrected the hypothesis ([post on X, July 28, 2026](#)) and a version of his theorem, in a bespoke proof calculus, has been proved in Lean [20].

APPENDIX A. GLOSSARY OF MACHINE LEARNING TERMS FOR MATHEMATICIANS

neural network: a function with many (up to trillions of) adjustable real parameters (its *weights*), built by composing simple layers and tuned by *gradient descent* to reduce a training *loss*.

layer: a map $x \mapsto \sigma(Wx + b)$: an affine map, whose matrix and vector entries are weights, followed by a fixed nonlinearity σ applied to each coordinate; a network is a composition of layers (a transformer alternates them with attention layers, which mix across positions).

neuron: one coordinate of a layer, the scalar function $x \mapsto \sigma(\langle w_j, x \rangle + b_j)$ read off row j of the layer’s matrix; in a transformer, a coordinate of one of its non-attention layers.

language model: a neural network trained to predict the next token of text; today’s chat systems are language models further trained to follow instructions.

token: the atomic unit (word or word-fragment) a language model reads and predicts.

- loss:** the real-valued function a network is trained to minimize; for language models, the negative log-probability it assigned to the actual next token.
- weights:** the trainable parameters of an artificial neural network: the entries of its matrices, trained by gradient descent and fixed at deployment time.
- activations:** the values that flow through a neural network on a given input (the outputs of its layers), as opposed to the fixed weights.
- activation space:** the copy of $\mathbb{R}^n$ in which a given layer’s activations live, one coordinate per neuron; feature directions are directions in this space.
- gradient descent:** the optimizer: repeatedly nudge the weights w against the gradient of the loss, $w \leftarrow w - \eta \nabla L(w)$.
- ReLU:** the “rectified linear unit” $x \mapsto \max(0, x)$, a canonical piecewise-linear nonlinearity; modern transformers often use smooth or gated variants instead (GELU, SiLU, SwiGLU).
- transformer:** the dominant neural network architecture for language: a stack of layers that mix information across token positions (each position selectively reads from the others) and read from / write to the residual stream.
- residual stream:** the running vector of activations a transformer reads from and writes to at each layer; a natural home for feature directions.
- feature:** a hypothesized variable or property a network represents in its activations (e.g. “the text is in French”).
- probe:** a simple (usually linear) function of a network’s activations, trained to read off some quantity of interest.
- policy:** an agent’s decision rule (a possibly randomized map from observations to actions).
- reinforcement learning:** training an agent by scoring its actions with a numerical *reward* and adjusting its policy to earn more of it.
- superposition:** a hypothesized way for a neural network to represent more features than it has dimensions (as non-orthogonal directions, tolerable when few features are active at once).
- sparse autoencoder:** a network trained to reconstruct another network’s activations as sparse combinations of learned dictionary directions; an empirical tool for extracting features.
- interpretability:** reverse-engineering a trained network into human-understandable structure (circuits, features, algorithms), validated by intervention and not by human-readable description alone.
- chain of thought:** the text a language model writes while reasoning toward its answer; readable by humans, and for now the main window onto an AI’s reasoning.
- activation steering:** changing a network’s behavior at runtime by adding a fixed vector to its activations, without retraining; see Section 4.
- gradual disempowerment:** the risk that humans lose influence over the institutions that shape our lives, by incremental delegation to AI systems we do not understand rather than by any sudden loss of control.

REFERENCES

- [1] Abbe, E., Boix-Adserà, E., and Misiakiewicz, T. (2023). SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. *Conference on Learning Theory (COLT)* 2023. [arXiv:2302.11055](#).
- [2] Aharon, M., Elad, M., and Bruckstein, A. M. (2006). On the uniqueness of overcomplete dictionaries, and a practical way to retrieve them. *Linear Algebra and its Applications* 416(1), 48–67.
- [3] Alon, N., Bloom, T. F., Gowers, W. T., Litt, D., Sawin, W., Shankar, A., Tsimmerman, J., Wang, V., and Wood, M. M. (2026). Remarks on the disproof of the unit distance conjecture. [arXiv:2605.20695](#).
- [4] Alquier, P., and Ridgway, J. (2020). Concentration of tempered posteriors and of their variational approximations. *Annals of Statistics* 48(3), 1475–1497. [arXiv:1706.09293](#).
- [5] Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., and Nanda, N. (2024). Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems* 37. [arXiv:2406.11717](#).
- [6] Armstrong, S. and Mindermann, S. (2018). Occam’s razor is insufficient to infer the preferences of irrational agents. *Advances in Neural Information Processing Systems* 31. [arXiv:1712.05812](#).
- [7] Barász, M., Christiano, P., Fallenstein, B., Herreshoff, M., LaVictoire, P., and Yudkowsky, E. (2014). Robust cooperation in the Prisoner’s Dilemma: program equilibrium via provability logic. [arXiv:1401.5577](#).
- [8] Bricken, T., et al. (2023). Towards monosemanticity: decomposing language models with dictionary learning. *Transformer Circuits Thread*, Anthropic. <https://transformer-circuits.pub/2023/monosemantic-features>.
- [9] Cai, T. T., and Zhang, A. (2014). Sparse representation of a polytope and recovery of sparse signals and low-rank matrices. *IEEE Transactions on Information Theory* 60(1), 122–132. [arXiv:1306.1154](#).
- [10] Candès, E. J., Romberg, J., and Tao, T. (2006). Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory* 52(2), 489–509.
- [11] Candès, E. J. (2008). The restricted isometry property and its implications for compressed sensing. *Comptes Rendus Mathématique* 346(9–10), 589–592.
- [12] Chizat, L., and Bach, F. (2020). Implicit bias of gradient descent for wide two-layer neural networks trained with the logistic loss. *Proceedings of Machine Learning Research* 125 (COLT 2020), 1305–1338.
- [13] Chughtai, B., Chan, L., and Nanda, N. (2023). A toy model of universality: reverse-engineering how networks learn group operations. *International Conference on Machine Learning* 2023. [arXiv:2302.03025](#).
- [14] Cooper, E., Oesterheld, C., and Conitzer, V. (2025). Characterising simulation-based program equilibria. *Proceedings of AAAI* 2025. [arXiv:2412.14570](#).
- [15] Critch, A. (2019). A parametric, resource-bounded generalization of Löb’s theorem, and a robust cooperation criterion for open-source game theory. *The Journal of Symbolic Logic* 84(4), 1368–1381. [arXiv:1602.04184](#).
- [16] Critch, A., Dennis, M., and Russell, S. (2022). Cooperative and uncooperative institution designs: surprises and problems in open-source game theory. [arXiv:2208.07006](#).
- [17] Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. [arXiv:2309.08600](#).
- [18] Dennett, D. C. (1987). *The Intentional Stance*. MIT Press.
- [19] Donoho, D. L. (2006). Compressed sensing. *IEEE Transactions on Information Theory* 52(4), 1289–1306.

- [20] Duclaux, C., Formenti, R., Cobben, P., Schölkopf, B., and Jin, Z. (2026). Proving your way to cooperation: formalizing proof-based open source game theory in Lean. *ICML 2026 Workshop on AI for Math*. <https://github.com/ColombanD/open-source-game-theory>.
- [21] Dyson, F. (2009). Birds and frogs. *Notices of the American Mathematical Society* 56(2), 212–223.
- [22] Eisenstat, S. (2025). Condensation: a theory of concepts. Manuscript. <https://www.sameisenstat.net/doc/condensation-25-07.pdf>.
- [23] Elhage, N., et al. (2022). Toy models of superposition. *Transformer Circuits Thread*. [arXiv:2209.10652](https://arxiv.org/abs/2209.10652).
- [24] Friston, K., Rigoli, F., Ognibene, D., Mathys, C., Fitzgerald, T., and Pezzulo, G. (2015). Active inference and epistemic value. *Cognitive Neuroscience* 6(4), 187–214.
- [25] Garfinkle, C. J., and Hillar, C. J. (2019). On the uniqueness and stability of dictionaries for sparse representation of noisy signals. *IEEE Transactions on Signal Processing* 67(23), 5884–5892. [arXiv:1606.06997](https://arxiv.org/abs/1606.06997).
- [26] Garraabrant, S., Benson-Tilsen, T., Critch, A., Soares, N., and Taylor, J. (2016). Logical induction. [arXiv:1609.03543](https://arxiv.org/abs/1609.03543).
- [27] Gowers, W. T. (2000). The two cultures of mathematics. In *Mathematics: Frontiers and Perspectives* (V. Arnold, M. Atiyah, P. Lax, and B. Mazur, eds.), American Mathematical Society, 65–78.
- [28] Greenblatt, R., Cotra, A., and Wijk, H. (2026). Brief independent investigation of agents’ behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. METR and Redwood Research, August 26, 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>.
- [29] Gribonval, R., Jenatton, R., and Bach, F. (2015). Sparse and spurious: dictionary learning with noise and outliers. *IEEE Transactions on Information Theory* 61(11), 6298–6319. [arXiv:1407.5155](https://arxiv.org/abs/1407.5155).
- [30] Grünwald, P. (2012). The safe Bayesian: learning the learning rate via the mixability gap. *Algorithmic Learning Theory (ALT)* 2012, 169–183.
- [31] Hadfield-Menell, D., Russell, S. J., Abbeel, P., and Dragan, A. (2016). Cooperative inverse reinforcement learning. *Advances in Neural Information Processing Systems* 29. [arXiv:1606.03137](https://arxiv.org/abs/1606.03137).
- [32] Hariharan, S., Birkbeck, C., Lee, S., Ma, H. K. G., Mehta, B., Poiroux, A., and Viazovska, M. (2026). Progress in formalizing sphere packing in dimension 8. [arXiv:2604.23468](https://arxiv.org/abs/2604.23468).
- [33] Hillar, C. J., and Sommer, F. T. (2015). When can dictionary learning uniquely recover sparse data from subsamples? *IEEE Transactions on Information Theory* 61(11), 6290–6297. [arXiv:1106.3616](https://arxiv.org/abs/1106.3616).
- [34] Hironaka, H. (1964). Resolution of singularities of an algebraic variety over a field of characteristic zero. *Annals of Mathematics* 79(1), 109–203; 79(2), 205–326.
- [35] Howard, J. V. (1988). Cooperation in the Prisoner’s Dilemma. *Theory and Decision* 24(3), 203–213.
- [36] Jacot, A., Gabriel, F., and Hongler, C. (2018). Neural tangent kernel: convergence and generalization in neural networks. *Advances in Neural Information Processing Systems* 31.
- [37] Johnson, W. B., and Lindenstrauss, J. (1984). Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics* 26, 189–206.
- [38] Khan, M. E., and Rue, H. (2023). The Bayesian learning rule. *Journal of Machine Learning Research* 24(281), 1–46. [arXiv:2107.04562](https://arxiv.org/abs/2107.04562).
- [39] Korbak, T., et al. (2025). Chain of thought monitorability: a new and fragile opportunity for AI safety. [arXiv:2507.11473](https://arxiv.org/abs/2507.11473).
- [40] Kulveit, J., Douglas, R., Ammann, N., Turan, D., Krueger, D., and Duvenaud, D. (2025). Gradual disempowerment: systemic existential risks from incremental AI development. [arXiv:2501.16946](https://arxiv.org/abs/2501.16946).

- [41] Kwa, T., West, B., Becker, J., et al. (2025). Measuring AI ability to complete long software tasks. METR report; *Advances in Neural Information Processing Systems* 38. [arXiv:2503.14499](#).
- [42] Lambert, N., et al. (2024). Tülu 3: pushing frontiers in open language model post-training. [arXiv:2411.15124](#).
- [43] Langosco, L., Koch, J., Sharkey, L., Pfau, J., and Krueger, D. (2022). Goal misgeneralization in deep reinforcement learning. *International Conference on Machine Learning* 2022. [arXiv:2105.14111](#).
- [44] Lau, E., Furman, Z., Wang, G., Murfet, D., and Wei, S. (2025). The local learning coefficient: a singularity-aware complexity measure. *Artificial Intelligence and Statistics (AISTATS)* 2025. [arXiv:2308.12108](#).
- [45] Löb, M. H. (1955). Solution of a problem of Leon Henkin. *The Journal of Symbolic Logic* 20(2), 115–118.
- [46] Maćkowiak, B., Matějka, F., and Wiederholt, M. (2023). Rational inattention: a review. *Journal of Economic Literature* 61(1), 226–273.
- [47] Maiya, S., Bartsch, H., Lambert, N., and Hubinger, E. (2025). Open character training: shaping the persona of AI assistants through Constitutional AI. [arXiv:2511.01689](#).
- [48] Marchetti, G. L., Hillar, C., Kragic, D., and Sanborn, S. (2024). Harmonics of learning: universal Fourier features emerge in invariant networks. *Conference on Learning Theory (COLT)* 2024. [arXiv:2312.08550](#).
- [49] The mathlib Community. (2020). The Lean mathematical library. *Certified Programs and Proofs (CPP)* 2020, 367–381. [arXiv:1910.09336](#).
- [50] Mei, S., Montanari, A., and Nguyen, P.-M. (2018). A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences* 115(33), E7665–E7671.
- [51] METR. (2026). Task-completion time horizons of frontier AI models. Updated May 8, 2026. <https://metr.org/time-horizons/>.
- [52] Millidge, B., Tschantz, A., and Buckley, C. L. (2021). Whence the expected free energy? *Neural Computation* 33(2), 447–482. [arXiv:2004.08128](#).
- [53] Murfet, D., Gordon, A., Adam, M., Wang, G., Hoogland, J., Baker, G., Snell, W., van Wingerden, S., Newgas, A., Snickers, B., and Hitchcock, R. (2026). Spectroscopy at scale: finding interpretable structure in Pythia-1.4B. Timaeus, research report. <https://timaeus.co/research/2026-04-21-spectroscopy-main/>.
- [54] Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. (2023). Progress measures for grokking via mechanistic interpretability. *International Conference on Learning Representations* 2023. [arXiv:2301.05217](#).
- [55] Ng, A. Y., and Russell, S. J. (2000). Algorithms for inverse reinforcement learning. *International Conference on Machine Learning* 2000, 663–670.
- [56] Oesterheld, C., and Conitzer, V. (2021). Safe Pareto improvements for delegated game playing. *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)* 2021; journal version, *Autonomous Agents and Multi-Agent Systems* 36 (2022).
- [57] OpenAI. (2026). An OpenAI model has disproved a central conjecture in discrete geometry. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- [58] Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35. [arXiv:2203.02155](#).
- [59] Pachocki, J. (2026). An alien mind. OpenAI, September 6, 2026. <https://openai.com/index/an-alien-mind/>.
- [60] Park, K., Choe, Y. J., and Veitch, V. (2024). The linear representation hypothesis and the geometry of large language models. *International Conference on Machine Learning* 2024. [arXiv:2311.03658](#).
- [61] Parr, T., Pezzulo, G., and Friston, K. J. (2022). *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press.

- [62] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press.
- [63] Pepin Lehalleur, S., Hoogland, J., Farrugia-Roberts, M., Wei, S., Gietelink Oldenziel, A., Wang, G., Carroll, L., and Murfet, D. (2025). You are what you eat: AI alignment requires understanding how data shapes structure and generalisation. [arXiv:2502.05475](#).
- [64] Rao, R. P. N., and Ballard, D. H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience* 2(1), 79–87.
- [65] Richens, J., and Everitt, T. (2024). Robust agents learn causal world models. *International Conference on Learning Representations* 2024. [arXiv:2402.10877](#).
- [66] Richens, J., Everitt, T., and Abel, D. (2025). General agents need world models. *International Conference on Machine Learning* 2025. [arXiv:2506.01622](#).
- [67] Rubinstein, A. (1998). *Modeling Bounded Rationality*. MIT Press.
- [68] Sawin, W. (2026). An explicit lower bound for the unit distance problem. [arXiv:2605.20579](#).
- [69] Saxe, A. M., McClelland, J. L., and Ganguli, S. (2014). Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *International Conference on Learning Representations* 2014. [arXiv:1312.6120](#).
- [70] Schelling, T. C. (1960). *The Strategy of Conflict*. Harvard University Press.
- [71] Sharkey, L., et al. (2025). Open problems in mechanistic interpretability. *Transactions on Machine Learning Research* 2025. [arXiv:2501.16496](#).
- [72] Tennenholtz, M. (2004). Program equilibrium. *Games and Economic Behavior* 49(2), 363–373.
- [73] Thurston, W. P. (1994). On proof and progress in mathematics. *Bulletin of the American Mathematical Society* 30(2), 161–177. [arXiv:math/9404236](#).
- [74] von Neumann, J., and Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.
- [75] Wagner, G. P. and Altenberg, L. (1996). Complex adaptations and the evolution of evolvability. *Evolution* 50(3), 967–976.
- [76] Watanabe, S. (2009). *Algebraic Geometry and Statistical Learning Theory*. Cambridge University Press.
- [77] Watanabe, S. (2013). A widely applicable Bayesian information criterion. *Journal of Machine Learning Research* 14, 867–897.
- [78] Wentworth, J., and Lorell, D. (2025). Natural latents: latent variables stable across ontologies. [arXiv:2509.03780](#).
- [79] Yudkowsky, E. and Soares, N. (2025). *If Anyone Builds It, Everyone Dies: Why Superhuman AI Would Kill Us All*. Little, Brown and Company.
- [80] Zhang, C., Bengio, S., Hardt, M., Recht, B., and Vinyals, O. (2017). Understanding deep learning requires rethinking generalization. *International Conference on Learning Representations* 2017. [arXiv:1611.03530](#).

DEPARTMENT OF MATHEMATICS, CORNELL UNIVERSITY, ITHACA, NY 14853

Email address: lionel.levine@cornell.edu

URL: lionellevine.github.io